

experiment-08

November 11, 2025

```
[1]: # --- Step 1: Import Required Libraries ---  
# (from exp 8.pdf, Cell 2)  
import pandas as pd  
from sklearn.datasets import load_iris  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import accuracy_score, precision_score, recall_score  
from sklearn.linear_model import LogisticRegression  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.svm import SVC  
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.naive_bayes import GaussianNB  
  
# --- Step 2: Load Dataset ---  
# (from exp 8.pdf, Cell 3)  
iris = load_iris()  
X = iris.data  
y = iris.target  
  
# --- Step 3: Split Data into Training and Testing Sets ---  
# (from exp 8.pdf, Cell 5)  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
                                                    random_state=42)  
  
# --- Step 4: Feature Scaling ---  
# (from exp 8.pdf, Cell 5)  
# The experiment scales the data and uses this scaled data for all models  
# to ensure a consistent comparison basis, especially for models  
# sensitive to feature scale (like LogReg, SVM, and KNN).  
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)  
  
# --- Step 5: Define Models ---  
# (from exp 8.pdf, Cell 6)  
models = {
```

```

    "Logistic Regression": LogisticRegression(max_iter=200),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "SVM": SVC(kernel='rbf', random_state=42),
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB()
}

# --- Step 6: Train and Evaluate Models ---
# (from exp 8.pdf, Cell 7)
results = []

for name, model in models.items():
    # Train the model on the scaled training data
    model.fit(X_train, y_train)

    # Make predictions on the scaled test data
    y_pred = model.predict(X_test)

    # Calculate metrics
    acc = accuracy_score(y_test, y_pred)
    # Use average='macro' for multiclass precision/recall
    prec = precision_score(y_test, y_pred, average='macro')
    rec = recall_score(y_test, y_pred, average='macro')

    results.append([name, round(prec, 2), round(rec, 2), round(acc, 2)])

# --- Step 7: Display Results ---
# (from exp 8.pdf, Cell 8)
df_results = pd.DataFrame(results, columns=["Algorithm", "Precision", "Recall", "Accuracy"])
df_results = df_results.sort_values(by="Accuracy", ascending=False)

print("\nComparison of Classification Algorithms:\n")
print(df_results)

```

Comparison of Classification Algorithms:

	Algorithm	Precision	Recall	Accuracy
0	Logistic Regression	1.0	1.0	1.0
1	Decision Tree	1.0	1.0	1.0
2	Random Forest	1.0	1.0	1.0
3	SVM	1.0	1.0	1.0
4	KNN	1.0	1.0	1.0
5	Naive Bayes	1.0	1.0	1.0

[]: